

浙江大学实验报告

专业：电气工程及其自动化

姓名：吴舟越

学号：3240103172

日期：2025.10.28

地点：东三 406

课程名称：电路与电子技术实验I 指导老师：张德华 成绩：_____

实验名称：PWM 电机调速控制 同组学生姓名：李言康

一. 实验目的

通过此次实验，我将深入理解 PWM 电机调速控制的原理及其应用，掌握时序逻辑电路的简易设计与实际应用，探究数字式 PWM 电路的基本架构及其实现方法。这将有助于我在前期智能小车的基础上成功增添调速功能。在此过程中，我还将进一步熟悉基于原理图设计的复杂 VHDL 语言编程流程，熟练运用 Quartus 软件进行数字电路仿真，并提升对示波器等常用电子仪器的操作能力。

二. 基本实验内容

(一) 应用 QUARTUS 软件设计、仿真 PWM 电路

1. 实验原理

1. 数字式 PWM 电路的基本结构如图 1 所示。其中，方波发生电路产生固定频率的方波信号，代表数字电路高电平“1”与低电平“0”的变化情况；N 进制计数器输出的波形为锯齿波，其大小随输入脉冲在 0 至 N-1 之间循环变化：每当检测到一个脉冲信号，计数器加 1，直至计数达到 N，便重新从 0 开始计数（如图 2 所示）。比较器的 A 端输入来自计数器的锯齿波信号，B 端输入外部开关设定的设定值。当 $A > B$ 时，比较器输出高电平“1”，反之输出低电平“0”。因此，最终输出波形为方波（如图 2 所示），且其占空比可通过设定值进行调节，从而实现 PWM 的逐档调节。若方波发生器的频率为 f ，则 PWM 波的频率等于 f/N ，PWM 波的占空比由比较设定值决定，共分为 N 档。

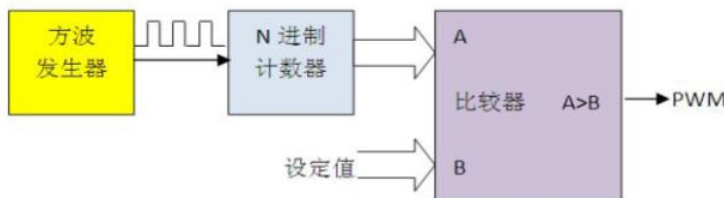


图 1

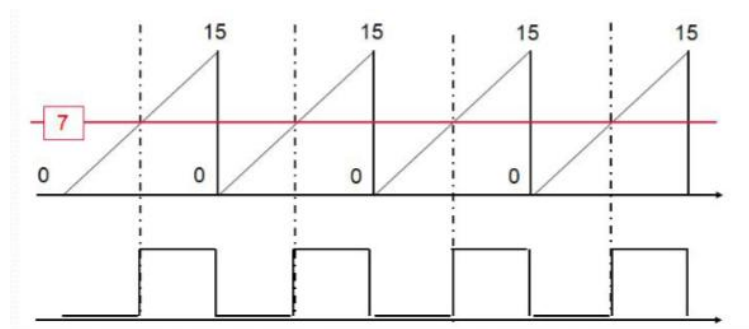


图 2

2. FPGA 实现 PWM 设计的框图如图 3 所示。FPGA 芯片能够输出固定频率为 50MHz 的方波信号，然而，由于该系统频率过高，不适合直接用于 PWM 调速电路，因此需要在计数器前加入分频器，将 50MHz 转换为 PWM 所需的频率。N 进制计数器用于产生锯齿波（建议采用 16 进制计数器，即 16 档），若计数器为 16 进制，则输出为 4 位二进制。比较器包含两个 4 位二进制比较单元。此外，比较器的设定值可通过 FPGA 板上的开关进行调节，无需反复编译并下载文件至 FPGA。

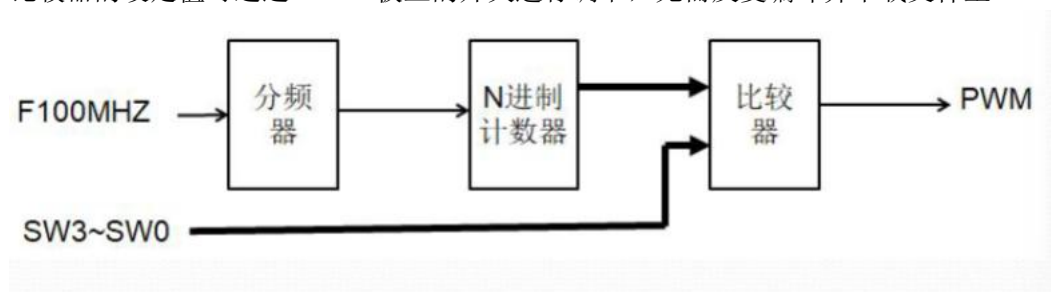


图 3

2. 主要仪器设备

Quartus 软件。

3. 测试方案与测试过程

一、完成 PWM 电路设计

在 Quartus 中新建工程，总体工程设计为一个主文件（上层）和四个子文件（底层），分别为分频器模块、16 进制计数器模块、比较器模块和开关控制分频器模块。文件夹结构如图 4 所示。

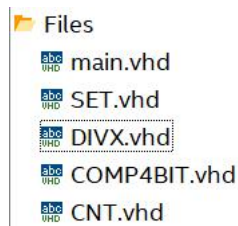


图 4

其中，主程序代码如图 5 所示，开关控制分频器（SET）模块代码如图 6 所示，分频器模块（DIVX）

代码如图 7 所示，比较器（COMP4BIT）代码如图 8 所示，16 进制计数器（CNT）代码如图 9 所示。

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5
6 entity main is
7     port (LED6,LED7: in std_logic;
8           R1: out std_logic;
9           clk0,rst: in std_logic;
10          PWM_in: in std_logic_vector(3 downto 0);
11          Div_set : in std_logic_vector(3 downto 0);
12          light: out std_logic_vector(3 downto 0);
13          PWM_out, PWM_out2: out std_logic);
14 end main;
15
16 architecture BEHAV of main is
17
18     Component CNT is
19         port (Clk,Clr_n: in std_logic;
20              Q: out std_logic_vector(3 downto 0);
21              CO: out std_logic);
22     end Component;
23
24     Component COMP4BIT is
25         port ( clk: in std_logic;
26              A,B: in std_logic_vector(3 downto 0);
27              AGB1,AGB2:out std_logic);
28     end Component;
29
30     Component DIVX is
31         port ( Freq_in,Clr_n: in std_logic;
32              Div_num:in natural range 0 to 49999999;
33              Freq_out: out std_logic);
34     end Component;
35
36     Component SET is
37         port ( Div: in std_logic_vector(3 downto 0);
38              num: out natural range 0 to 200000);
39     end Component;
40
41     signal D: std_logic_vector(1 downto 0);
42     signal clk_temp,clk_temp2: std_logic;
43     signal A_temp: std_logic_vector(3 downto 0);
44     shared variable Div_num_set : natural range 0 to 200000;
45
46 begin
47     U1: SET port map (Div=>Div_set,num=>Div_num_set);
48     U2: DIVX port map (Freq_in=>clk0, Clr_n=>rst, Div_num=>Div_num_set, Freq_out=>clk_temp);
49     U3: CNT port map (clk=>clk_temp, Clr_n=>rst, Q=>A_temp, CO=>clk_temp2);
50     U4: COMP4BIT port map (clk=>clk_temp2, A=>A_temp, B=>PWM_in, AGB1=>PWM_out, AGB2=>PWM_out2);
51
52     D<= LED7 & LED6;
53     light<=PWM_in;
54     process(D)
55     begin
56         case D is
57             when "00" => L<='0';R<='0';
58             when "01" => L<='1';R<='0';
59             when "10" => L<='0';R<='1';
60             when "11" => L<='1';R<='1';
61             when others => null;
62         end case;
63     end process;
64 end BEHAV;
65
66

```

图 5

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5
6 entity SET is
7     port(Div: in std_logic_vector(3 downto 0)
8         num: out natural range 0 to 200000);
9 end SET;
10
11 architecture BEHAV of SET is
12 begin
13     process(Div)
14     begin
15         case Div is
16             when "0000"=>num<=1;
17             when "0001"=>num<=100;
18             when "0010"=>num<=400;
19             when "0011"=>num<=800;
20             when "0100"=>num<=1200;
21             when "0101"=>num<=2000;
22             when "0110"=>num<=3000;
23             when "0111"=>num<=6000;
24             when "1000"=>num<=8000;
25             when "1001"=>num<=10000;
26             when "1010"=>num<=12500;
27             when "1011"=>num<=15000;
28             when "1100"=>num<=20000;
29             when "1101"=>num<=30000;
30             when "1110"=>num<=40000;
31             when "1111"=>num<=80000;
32         end case;
33     end process;
34 end BEHAV;
35

```

图 6

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5
6 entity DIVX is
7     port(Freq_in,Clr_n : in std_logic;
8          Div_num : in natural range 0 to 49999999;
9          Freq_out : out std_logic);
10 end DIVX;
11
12 architecture BEHAV of DIVX is
13 begin
14     process (Freq_in,Clr_n)
15         variable cnt_val : natural range 0 to 49999999;
16     begin
17         if(Clr_n = '0') then
18             cnt_val := 0;
19         elsif(Freq_in'event and Freq_in = '1') then
20             if(cnt_val = Div_num-1) then
21                 cnt_val := 0;
22             elsif(cnt_val < Div_num/2) then
23                 cnt_val := cnt_val+1;
24                 Freq_out <= '0';
25             else
26                 cnt_val := cnt_val+1;
27                 Freq_out <= '1';
28             end if;
29         end if;
30     end process;
31 end BEHAV;
32

```

图 7

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5
6 entity COMP4BIT is
7     port(clk: in std_logic;
8          A,B: in std_logic_vector(3 downto 0);
9          AGB1,AGB2: out std_logic);
10 end COMP4BIT;
11
12 architecture BEHAV of COMP4BIT is
13     signal B_temp: std_logic_vector(3 downto 0);
14 begin
15     process(A,B,clk)
16     begin
17         if (clk 'event and clk = '1') then
18             B_temp <= B;
19         end if;
20         if (A >= B_temp) then
21             AGB1 <= '1';
22             AGB2 <= '1';
23         else
24             AGB1 <= '0';
25             AGB2 <= '0';
26         end if;
27     end process;
28 end BEHAV;
29

```

图 8

```

1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.std_logic_arith.all;
4 use IEEE.std_logic_unsigned.all;
5
6 entity CNT is
7     port(Clk,Clr_n: in std_logic;
8          Q: out std_logic_vector(3 downto 0);
9          CO: out std_logic);
10 end CNT;
11
12 architecture BEHAV of CNT is
13     signal Q_tmp: std_logic_vector(3 downto 0);
14 begin
15     CO <= '1' when (Q_tmp = "1111") else '0';
16     process(Clk,Clr_n)
17     begin
18         if(Clr_n = '0') then
19             Q_tmp <= "0000";
20         elsif(Clk 'event and Clk = '1') then
21             Q_tmp <= Q_tmp+1;
22         end if;
23     end process;
24     Q <= Q_tmp;
25 end BEHAV;
26

```

图 9

二、仿真 PWM 电路

各子程序的仿真结果如下所示：

（一）分频器 DIVX:

1. 如图 10 所示修改代码，将 DIV_NUM 从 entity 的变量修改为 architecture 中的变量，由于 entity 中变量只接受硬件输入，修改为 architecture 中变量后可以对其直接赋值以便仿真。这里将 DIV_NUM 赋值为 10，表示分频器每 10 个输入时钟周期产生一个输出周期，实现 10 分频。
2. 先将文件 DIVX.vhd 设为 Top_level entity，之后编译文件，进行仿真。设置 Grid Size 为 10ns, Clr_n 恒为高电平，Freq_in 周期为 40ns，初始电平为低电平，运行仿真。

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
entity DIVX is
port(Freq_in,Clr_n : in std_logic;
      Freq_out : out std_logic);
end DIVX;

architecture BEHAV of DIVX is
constant DIV_NUM:natural:=10;
begin
process (Freq_in,Clr_n)
variable cnt_val : natural range 0 to 49999999;
begin
    if(Clr_n = '0') then
        cnt_val := 0;
    elsif(Freq_in'event and Freq_in = '1') then
        if(cnt_val = Div_num-1) then
            cnt_val := 0;
        elsif(cnt_val < Div_num/2) then
            cnt_val := cnt_val+1;
            Freq_out <= '0';
        else
            cnt_val := cnt_val+1;
            Freq_out <= '1';
        end if;
    end if;
end process;
end BEHAV;
```

图 10

（二）16 进制计数器 CNT

先将文件 CNT.vhd 设为 Top_level entity，之后编译文件，进行仿真。设置 Grid Size 为 10ns, Clk 周期为 20ns, Clk_n 周期为 640ns，初始电平为低电平，运行仿真。

（三）四位二进制比较器 COMP4BIT

为了独立测试比较器，如图 11 修改代码，再将文件 COMP4BIT.vhd 设为 Top_level entity，随后编译并启动仿真。设置 Grid Size 为 10ns，输入信号 B 赋值为“1000”，输入信号 A 设置 Count number 从 0000 每隔 10ns 递增 1，直到 1111 后回到 0000，设置如图 11 所示。

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
entity COMP4BIT is
    Port(
        A,B: in std_logic_vector(3 downto 0);
        AGB: out std_logic);
end COMP4BIT;

architecture BEHAV of COMP4BIT is
begin
    process(A,B)
    begin
        if (A>B) then
            AGB <= '1';
        else
            AGB <= '0';
        end if;
    end process;
end BEHAV;
```

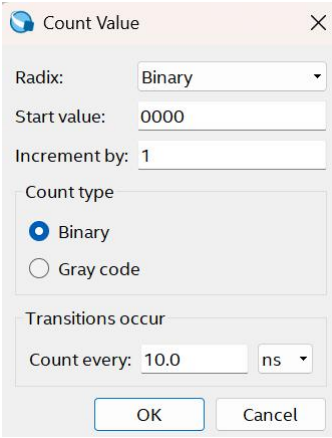


图 11

4. 结果记录与分析

各子程序的仿真结果如下所示：

(一) 分频器 DIVX:

仿真结果如图 12 所示。仿真波形显示，Freq_out 周期 400ns，恰好为 Freq_in 周期的 10 倍，验证了分频功能正确实现。

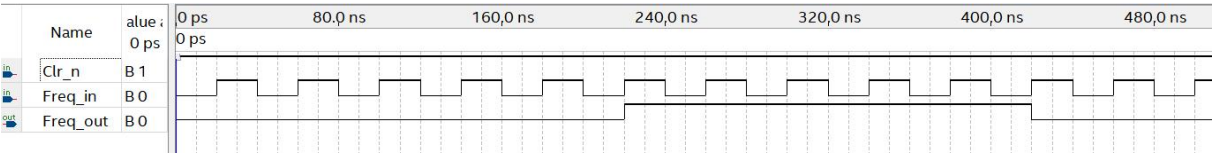


图 12

(二) 16 进制计数器 CNT

仿真结果如图 13 所示。仿真波形显示，Clk 每 20ns 产生一个上升沿，当 CLr_n 为高电平时，计数器从 0 开始递增，16 个时钟周期完成一次循环，计数器当前值 Q 从 0000 递增至 1111 后清零，符合 4 位二进制计数规律，当 Q 达到 1111 时，CO（进位输出）为 1，其余时刻 CO 为 0，表明进位信号在计数溢出时正确产生。当 Clk_n 下降沿到来时，Q 同步清零，验证了异步复位功能的可靠性。

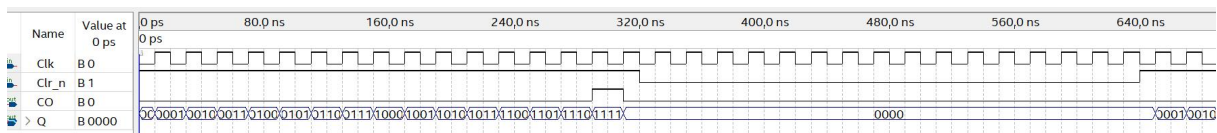


图 13

(三) 四位二进制比较器 COMP4BIT

仿真结果如图 14 所示。仿真波形显示，输入信号 A 从 0000 开始每隔 10ns 递增 1，当 A 小于 B（1000）时，输出 AGB 为低电平；当 A 等于 B 时，AGB 跳变高电平；当 A 大于 B 时，AGB 保持高电平，直至 A 再次小于 B 时回落。波形清晰表明比较器能准确识别 A 与 B 的大小关系，输出响应及时，逻辑功能完全符合设计预期。

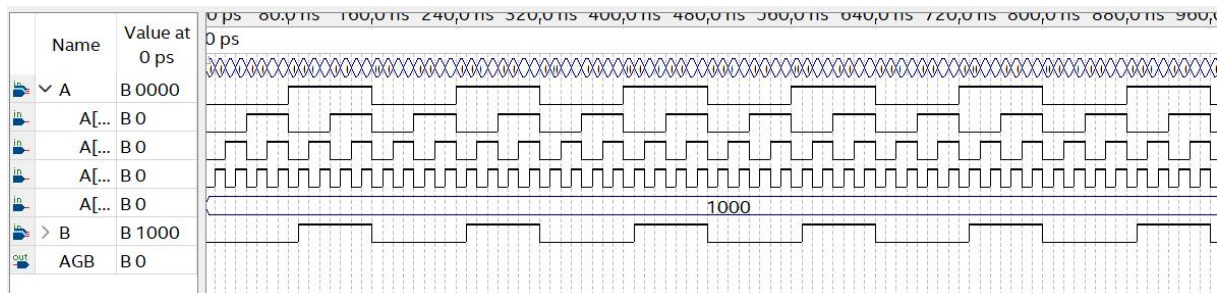


图 14

(二) 基于 FPGA 开发板用 QUARTUS 软件综合、布线并产生并生成下载文件

1. 实验原理

查阅资料可知,FPGA 50MHz 信号、GPIO 及 SW9~SW0、LED9~LED0 引脚如图 15 所示。

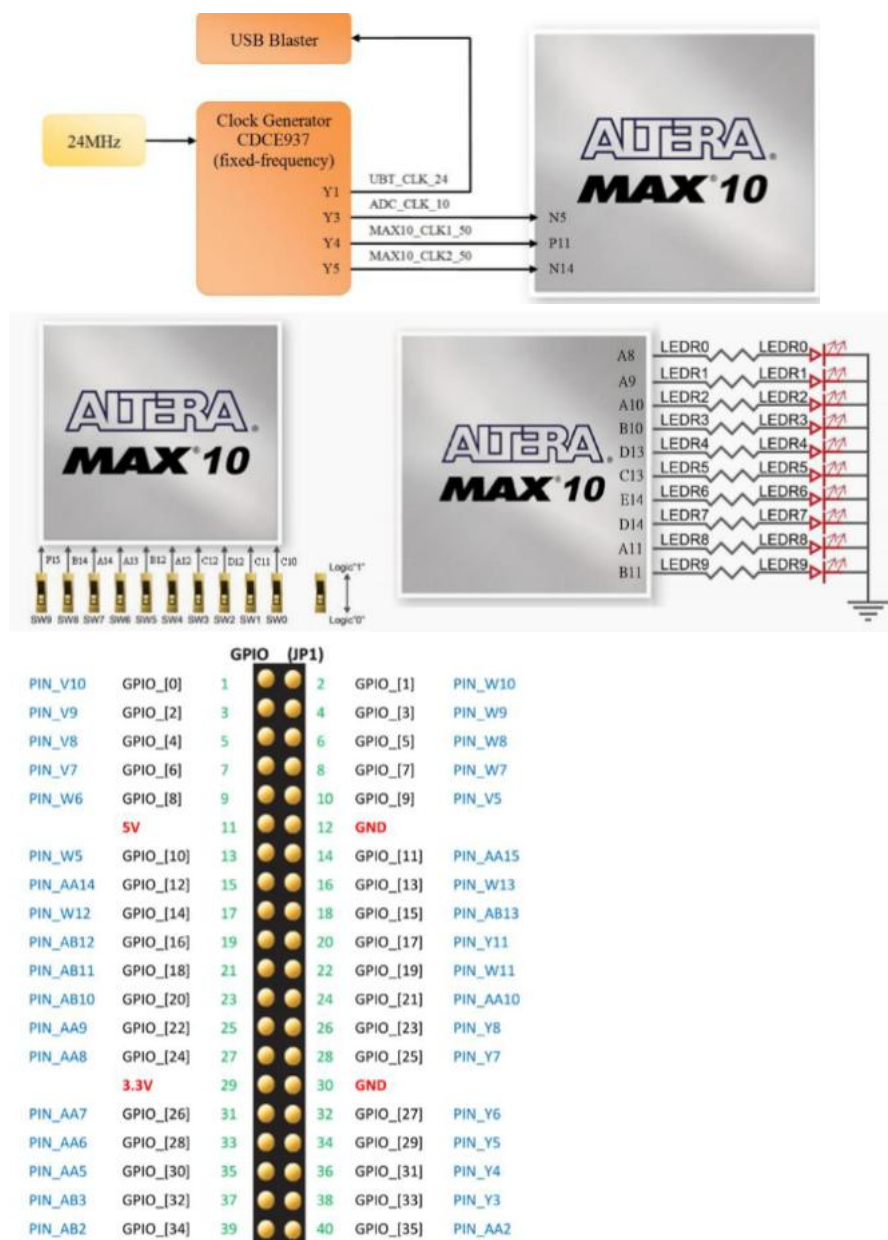


图 15

2. 主要仪器设备

Quartus 软件

3. 测试方案与测试过程

设定 main.vhd 为 top-level entity，先进行一次编译。

之后根据图 15 进行引脚分配如图 16 所示，再次编译。

in	clk0	Input	PIN_P11	3	B3_N0	PIN_P11	2.5 V		12mA...ult)		
in	Div_set[3]	Input	PIN_F15	7	B7_N0	PIN_F15	2.5 V		12mA...ult)		
in	Div_set[2]	Input	PIN_B14	7	B7_N0	PIN_B14	2.5 V		12mA...ult)		
in	Div_set[1]	Input	PIN_A14	7	B7_N0	PIN_A14	2.5 V		12mA...ult)		
in	Div_set[0]	Input	PIN_A13	7	B7_N0	PIN_A13	2.5 V		12mA...ult)		
out	L	Output	PIN_Y6	3	B3_N0	PIN_Y6	2.5 V		12mA...ult)	2 (default)	
in	LED6	Input	PIN_W9	3	B3_N0	PIN_W9	2.5 V		12mA...ult)		
in	LED7	Input	PIN_W10	3	B3_N0	PIN_W10	2.5 V		12mA...ult)		
out	light[3]	Output	PIN_B10	7	B7_N0	PIN_B10	2.5 V		12mA...ult)	2 (default)	
out	light[2]	Output	PIN_A10	7	B7_N0	PIN_A10	2.5 V		12mA...ult)	2 (default)	
out	light[1]	Output	PIN_A9	7	B7_N0	PIN_A9	2.5 V		12mA...ult)	2 (default)	
out	light[0]	Output	PIN_A8	7	B7_N0	PIN_A8	2.5 V		12mA...ult)	2 (default)	
in	PWM_in[3]	Input	PIN_C12	7	B7_N0	PIN_C12	2.5 V		12mA...ult)		
in	PWM_in[2]	Input	PIN_D12	7	B7_N0	PIN_D12	2.5 V		12mA...ult)		
in	PWM_in[1]	Input	PIN_C11	7	B7_N0	PIN_C11	2.5 V		12mA...ult)		
in	PWM_in[0]	Input	PIN_C10	7	B7_N0	PIN_C10	2.5 V		12mA...ult)		
out	PWM_out	Output	PIN_Y5	3	B3_N0	PIN_Y5	2.5 V		12mA...ult)	2 (default)	
out	PWM_out2	Output	PIN_Y3	3	B3_N0	PIN_Y3	2.5 V		12mA...ult)	2 (default)	
out	R	Output	PIN_Y4	3	B3_N0	PIN_Y4	2.5 V		12mA...ult)	2 (default)	
in	rst	Input	PIN_Y7	3	B3_N0	PIN_Y7	2.5 V		12mA...ult)		

图 16

编译成功后生成 sof 下载文件。打开 programmer, 选择 main.sof。选择硬件为 ‘USB-Blaster’，点击 ‘Start’ 按钮，将程序烧录至芯片。如图 17 所示。

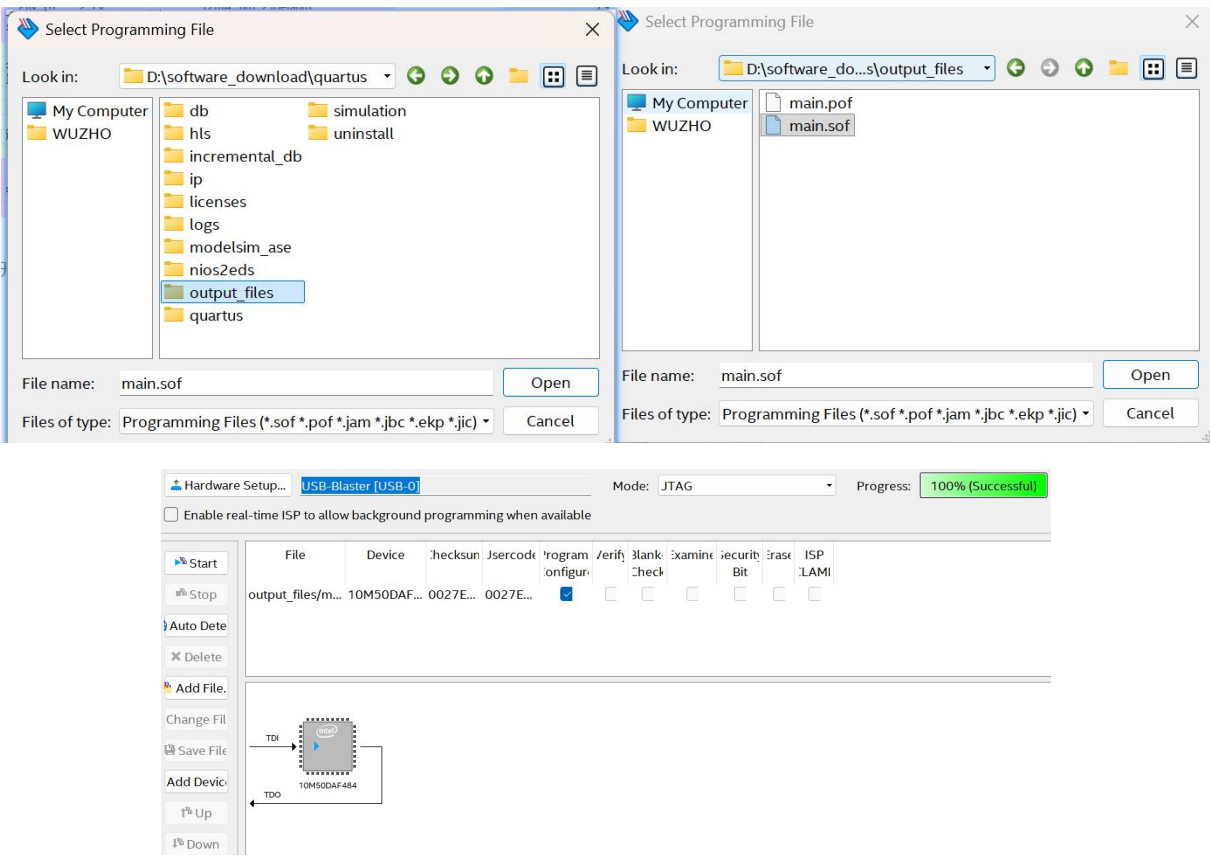


图 17

4. 结果记录与分析

成功生成下载文件，并成功烧入到 FPGA 芯片中。功能验证将在后续实验进行

（三）下载数字 PWM 产生电路，通过按键设定多种占空比，用示波器观察并记录 PWM 的输出波形，测量其周期、幅度和占空比。

1. 实验原理

通过实验二，成功将数字 PWM 产生电路下载至 FPGA 开发板。可以通过开关按键改变输入分频数。分频数控制输入方波频率。比较器的比较值改变 PWM 信号的占空比。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S、数字示波器 DS0X1102G

3. 测试方案与测试过程

设定分频数 $Div_num=2000$ ，对应开关状态 0101。实体开关设置如图所示

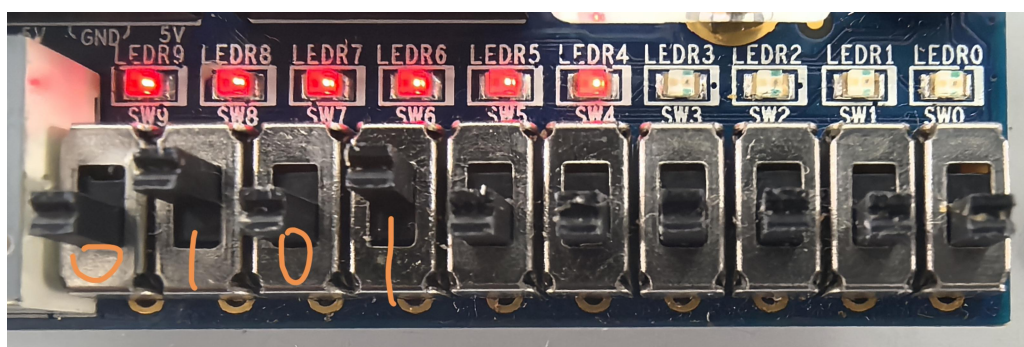
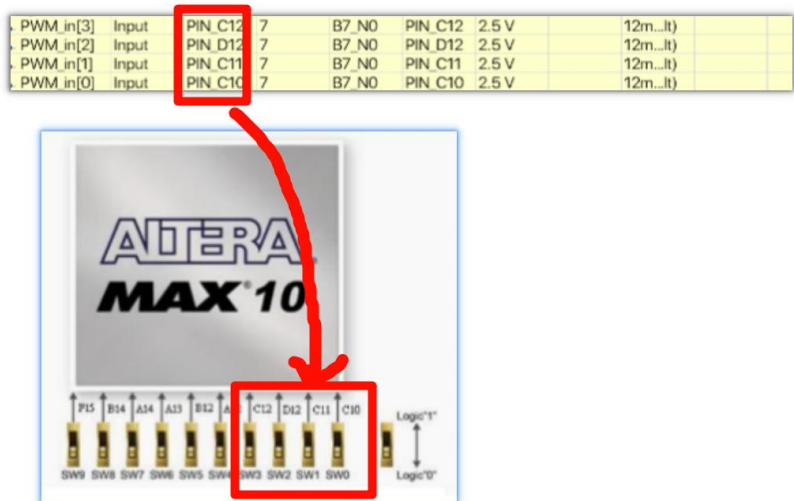


图 18

将 FPGA 上 PWM_out 对应的 PIN_Y5 引脚连接至示波器 CH1 通道的探头正极，接地端接开发板 GND。

如图 19 所示，调节 PWM_in 对应的开关 SW0 到 SW4。0000、0100、1000、1100、1110 分别对应占空比分别为 100%、75%、50%、25%、12.5%。调节开关开闭，通过示波器 Meas 功能测量



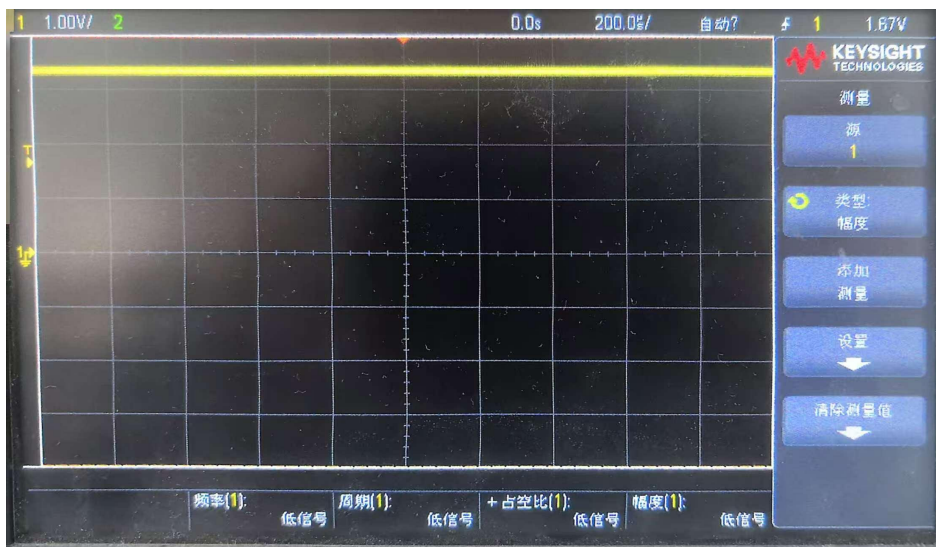
周期、幅度、占空比各参数。

图 19

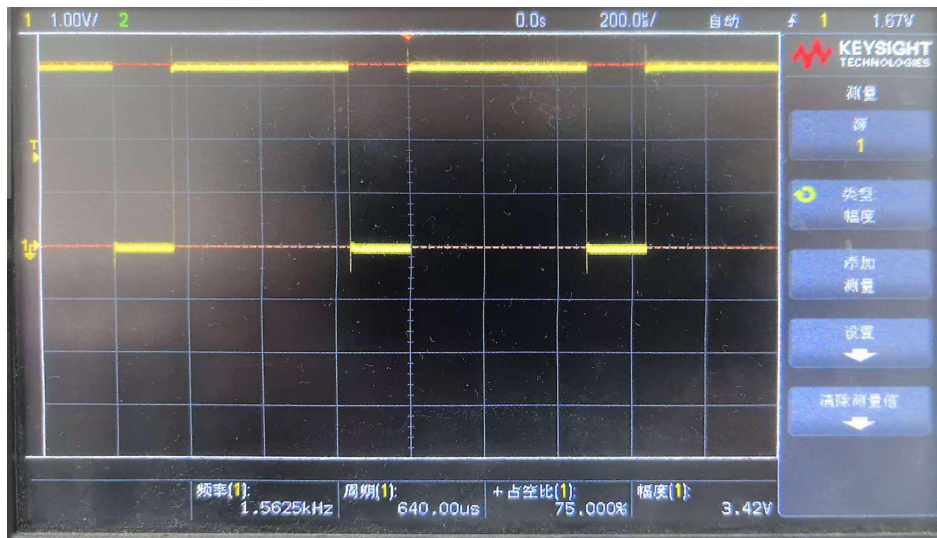
4. 结果记录与分析

记录实验结果如下，各参数在示波器界面低端栏中显示。

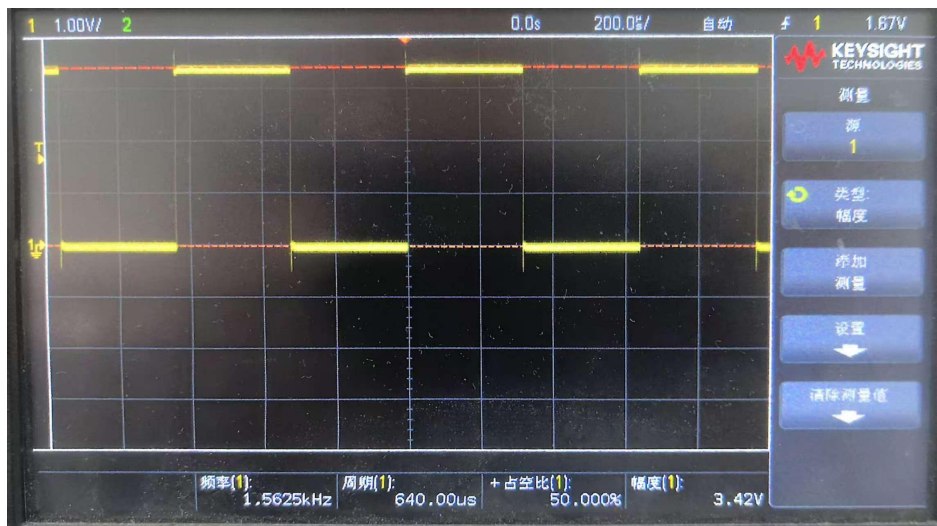
1. 占空比 100%：无周期性



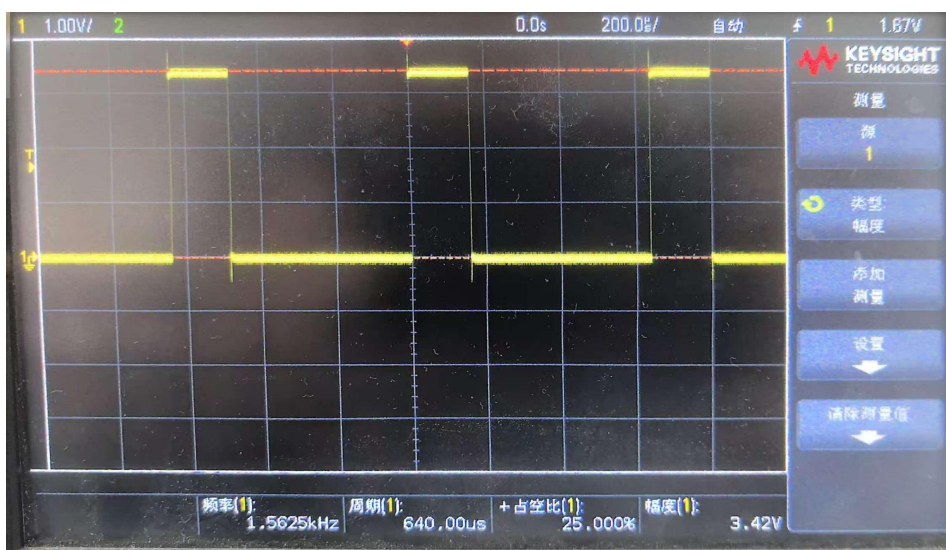
2. 占空比 75%:



3. 占空比 50%:



5. 占空比 25%:



综上，可以得出结论，占空比的改变通过调节输入开关状态实现，且与预设值一致。示波器测量结果清晰显示，PWM 波形的周期稳定在 640μs，幅度恒定为 3.42V，说明周期幅度与占空比无关。

进行理论结算：方波发生器频率 50MHz，分频数 Div_num=2000 时，实际输出频率为 $50\text{MHz}/2000=25\text{kHz}$ ，对应周期为 $40\text{ }\mu\text{s}$ 。经过 16 进制计数器计数实现 16 倍分频，最终输出周期为 $40*16=640\text{ }\mu\text{s}$ ，与实测结果一致。

（四）将 PWM 输出与电机驱动模块的总控输入 ENA 连接，
IN1=1, IN2=0，观察不同占空比时电机的运动速度

1. 实验原理

PWM 电路的输出波形占空比越低，意味着在一个完整周期内高电平的持续时间相对较短，而低电平的持续时间相应增长。这种调节方式会显著降低输出电压或电流在一个周期内的有效值，从而削弱对电机的驱动能力。随着有效值的减小，电机所获得的平均功率下降，导致其输出扭矩和旋转速度逐渐降低。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S

3. 测试方案与测试过程

设定分频数 Div_num=2000。设置稳压源输出+18VDC 至小车驱动板，将 FPGA、驱动板、检测板各引脚连接，如图 20 所示。分别调节 FPGA 板上 SW3~SW0 至 0100、1000、1100，即设定占空比分别为 75%、50%、25%，观察并记录 1min 内小车轮胎转速。

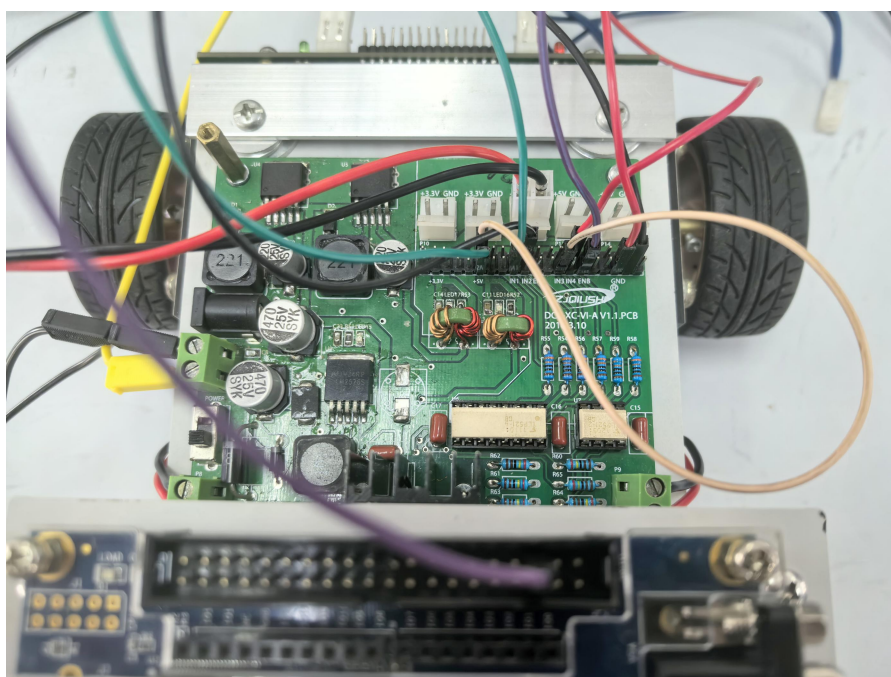


图 20

4. 结果记录与分析

记录不同占空比下小车轮胎的转速数据如表一所示。

表一：不同占空比下小车轮胎的转速数据

占空比	1min 内圈数	转速 (r/min)
25%	20	20
50%	49	49
75%	76	76

根据表一数据可知，随着占空比从 25%增至 75%，小车轮胎转速由 20r/min 提升至 76r/min，呈正相关趋势。这验证了 PWM 调制中占空比与电机平均输入功率的正相关，进而直接决定了电机转速的变化规律。

（四） 分频器模块通过三种不同的频率，测试并观察 “欠调，正常，过调” 电机工作情况。

1. 实验原理

对于调频而言，欠调是指调制信号的频率低于载波频率，而过调则是指调制信号的频率高于载波频率。欠调会放大低频信号，压缩高频信号，从而减小信号失真和噪声，但会降低信号的带宽利用率；而过调则会截断高频信号，仅允许低频信号通过传输，这虽然可以提高信号的带宽利用率，但会放大信号失真和噪声。在 PWM 调速电路中，选择合适的频率至关重要，因为当占空比改变时，速度变化显著，而欠调和过调的变化均不明显，无法有效实现调速。

2. 主要仪器设备

智能小车、稳压电源 GPD-4303S、数字万用表 UT890D+

3. 测试方案与测试过程

通过 SW9~SW6 调节分频数 Div_num 分别为 1、2000、80000，调节方法如图 21 所示。由 $f=50\text{MHz}/\text{Div_num}$ 可计算得 PWM 频率分别为 5MHz、25kHz、625Hz。

在频率确定的情况下调节占空比从 25%到 75%，观察并记录不同频率下小车的转速变化。

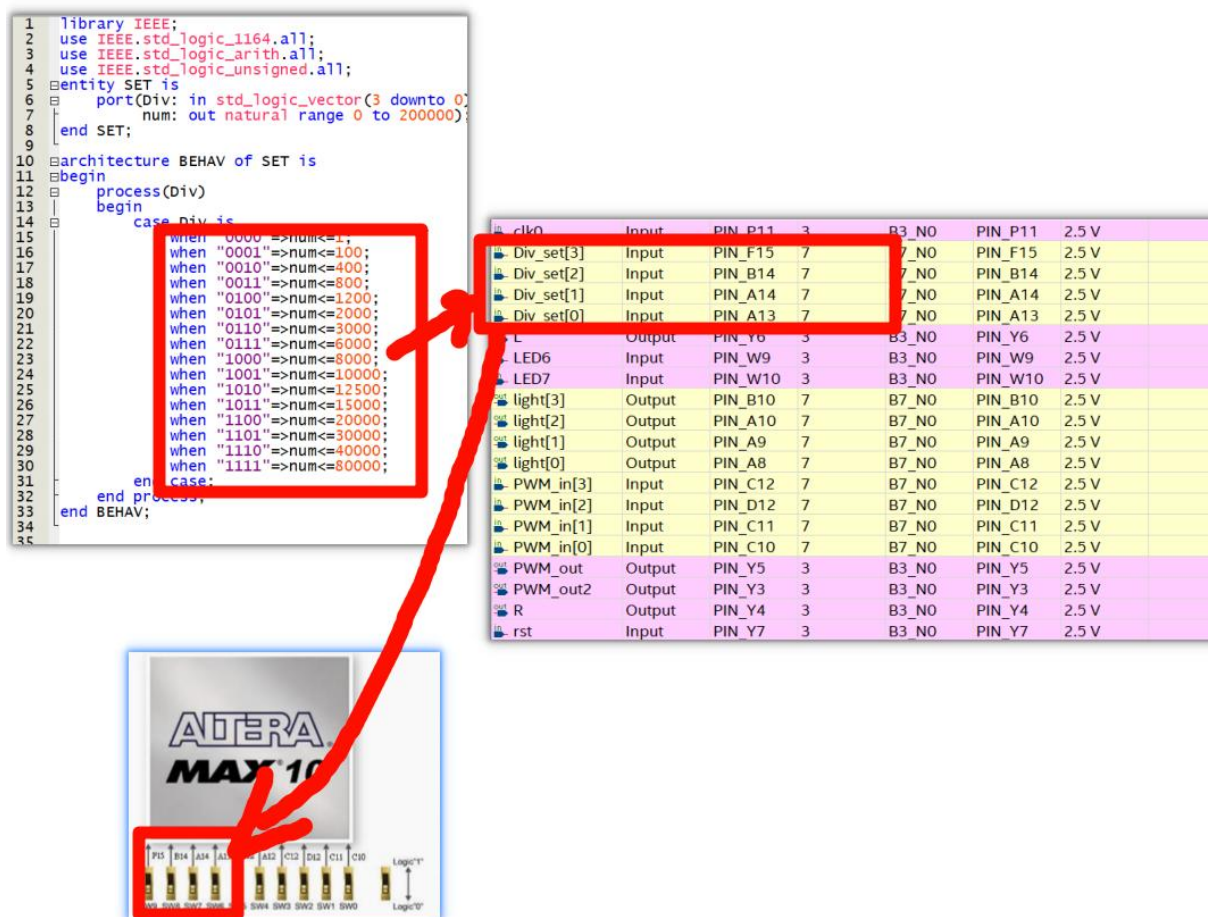


图 21

4. 结果记录与分析

记录数据如表 3 所示。由表 3 可知，在不同 PWM 频率下，随着占空比的增加，小车转速均呈上升趋势，但变化幅度受频率影响显著。当频率为 625Hz（Div_num=80000）时，转速响应迟缓，且电机存在明显脉动，表现为“欠调”现象，电机运行不平稳；当频率升至 25kHz（Div_num=2000）时，转速随占空比调节线性度高，运行平稳，属于“正常调制”区间；而当频率达到 5MHz（Div_num=1）时，电机几乎转速变化，出现“过调”现象。

推测原因为：高频下开关器件的响应能力受限，导致 PWM 信号无法有效被电机执行，造成调速失效。低频下转速响应迟缓且伴有明显抖动，因电机电枢惯性无法及时跟随脉冲变化，导致转矩波动加剧。

表 3：不同频率下小车转速随占空比变化数据

占空比	转速 (r/min)		
	频率		
	5MHz	25kHz	625Hz
25%	77	20	60
50%	77	49	69
75%	78	76	76

三、实验讨论与心得

1. 实验收获

- (1) 通过实验，掌握了 PWM 调速的基本原理及占空比、频率对电机转速的影响规律。实践表明，占空比越大，电机平均电压越高，转速越快，调速效果显著；而频率过高或过低均会影响控制精度与响应速度，需选择适中频率以兼顾效率与稳定性。
- (2) 同时，熟悉了 FPGA 开发板对 PWM 信号的生成与调节方法，掌握了通过硬件描述语言编程实现 PWM 波形输出的技术路径，并能灵活调整参数以适配不同电机的驱动需求。
- (3) 进一步加强了对代码中变量、软件虚拟引脚、硬件中真实引脚三者关系的理解，能够理解代码并通过真实引脚实现代码功能。

2. 实验细节

- (1) 占空比的调节只适用于“启动-停止”模式切换，不适用于“正转-反转”模式切换（可能导致电机损坏）
- (2) 分频器作用后的方波频率不能太高也不能太低。如果频率太高，会导致电机响应滞后，控制精度下降；若频率过低，则脉冲信号明显可感，造成转速不稳、震动加剧。
- (3) 依然小心接触不良问题。本人实验过程中一半以上错误都是接触不良造成的。